Specifications and Modeling I

- Writing specifications, and modeling, is **the process of developing documents that describe system or product intent and constraints**, and their translation into a variety of executable and declarative models.
- The top-level specifications and requirements are usually intentionally written in **a natural language**, rather than in a synthetic language.
- **Sufficient ambiguity** must remain in each specification or model to allow the model at the next level of refinement to meet its functional (and non-functional) requirements **without constraining it to a particular implementation**.
- Once a specification having an appropriate **balance of precision and ambiguity** is written, it serves as the basis for pre-partitioning analysis.
- The specification **cannot be written in the absence of implementation concerns** because it is clearly possible to specify requirements that cannot be implemented.

Specifications and Modeling II

- **The ambiguity must be preserved at each stage** in order to truly explore **the space of design alternatives**. Any "middle-out" design methodology here, in which there is a desire to reuse large amounts of existing IP - designs, platforms, libraries, and operating environment models - has a major impact on the specification of the system and its division into sub-specifications.
- Although **hardware and software** have traditionally been developed somewhat **in isolation** from one another, with software requiring a relatively stable hardware platform before software development proceeds, **this is no longer possible in an ESL flow**.
- The objective is **to develop hardware and software concurrently** to avoid late stage integration surprises.
- **Platforms** are used to manage the complexity of modern designs by employing **pre-verified hardware blocks and software modules**.

Specifications and Modeling III

- In an ideal world, software models should be written first so that pre-partitioning analysis can be used to define **the hardware requirements of the software**.
- In addition to the natural language specification, there is a **executable specifications** class:
  - Systep Behavioral Description
  - Run as a computer program
  - An opaque box perspective
- The specifications that fall into this class are the **executable architecture** specification and **executable design** specification.
- **The executable architecture specification** serves as a vehicle for **exploring design alternatives and demonstrating design concepts**. It may be written at various abstraction levels.

Specifications and Modeling IV

- **The executable design specification**, on the other hand, reflects **key microarchitecture structural decisions and exhibits high-level clear box behaviors**. Transaction-level modeling may be used in the creation of an executable specification to gain substantial performance.

Specification and Modeling Problems

- Capturing and Management of Requirements is **big problem**, especially when the project is large.
- There are several **different domains** in which ESL specifications is used and they needed **different approach** to specification and modeling.
- Specification should not **constraint the imeplmentation** but it will be good if it is somehow expressed **formally**.
- Problems with specification:
  - Requirements Management and Paper Specifications
  - ESL Domains
  - Executable Specifications
  - ESL Languages for Specification

Requirements Management and Paper Specifications I

- Requirement is a:
  - **Condition or capability needed by a user** to solve a problem or achieve an objective;
  - Condition or capability that must be met or possessed by a system or a system component to satisfy a **contract, standard, specification, or other formally imposed document**;
  - **Documented representation** of a condition or capability as above.
- Requirements emerge from **the problem domain**. They are used to describe the needs of customers. A requirement represents the customer's (the higher-level entity's) **understanding of a need**.

Requirements Management and Paper Specifications II

- During product line development, **a model of requirements** can be constructed and used to make selections to generate a new product. It can be helpful **to organize the model as a forest**, in which the requirements are related to each other in **parent–child relationships**. A requirement that is considered to elaborate on another requirement can be made a child of that requirement. This reflects many requirements document structures.
- **Requirements management** is a process that takes care of making **all requirements visible and traceable**. The requirements need to be tracked through the design process so that they are approved at the right level of decision making and followed up in appropriate design reviews. A requirement management system depends on the size and complexity of the organization, but generally placing trust only in paper documents will not suffice. **Some database automation is required**.

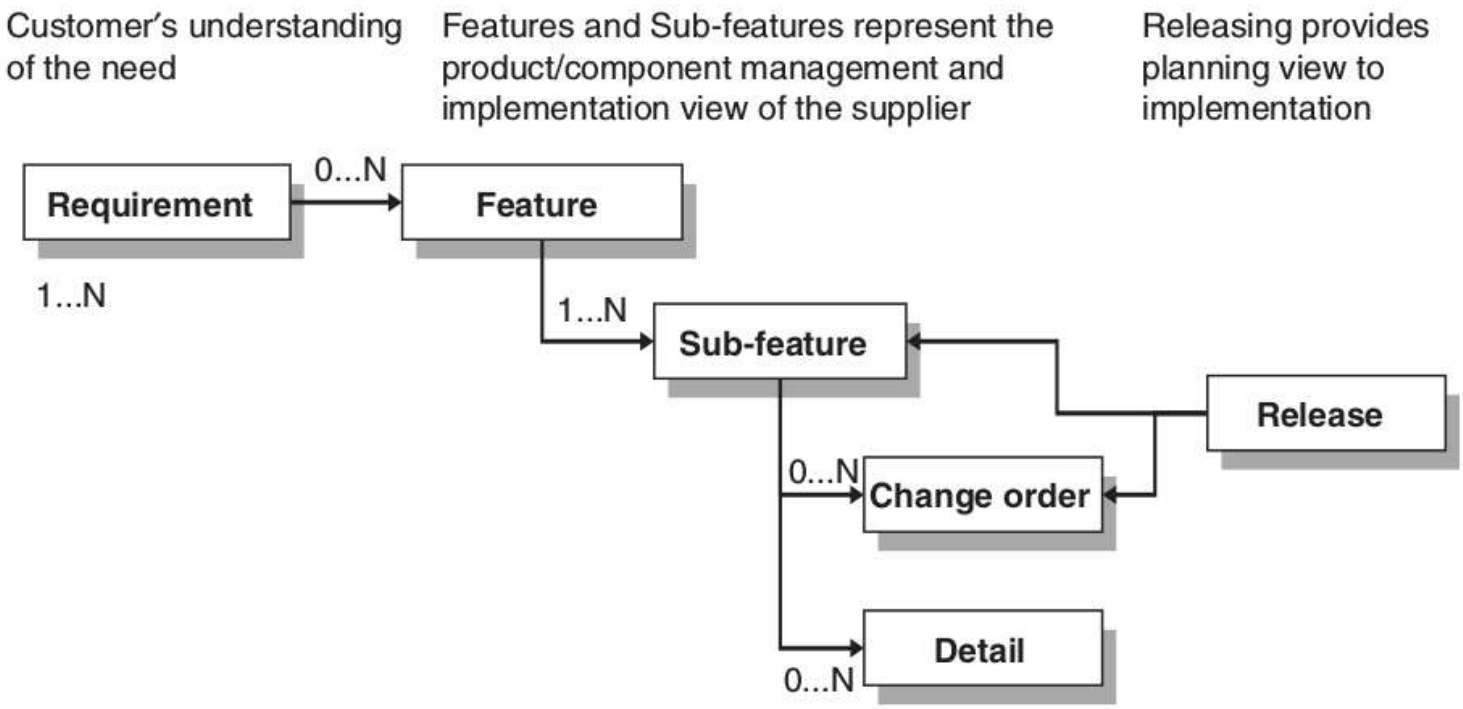
Requirements Management and Paper Specifications III

- A typical telecommunication product involves **thousands of individual requirements** that come from customers, standards, implementation technologies, and other sources. The nature of these requirements is diverse. One requirement may state that the product must have a specific type of voice codec, another that all voice processing is to be implemented with a specific DSP processor, and yet another specifies the microphone amplifier electrical characteristics. Each of these requirements is related to **the same end-user functionality**, but requires capturing **different kinds of information** from different sources.
- Each requirement affects different implementation teams or groups. Eventually, some requirements can be described **formally**, some can be made **executable models**, and some can only be in **natural language**.

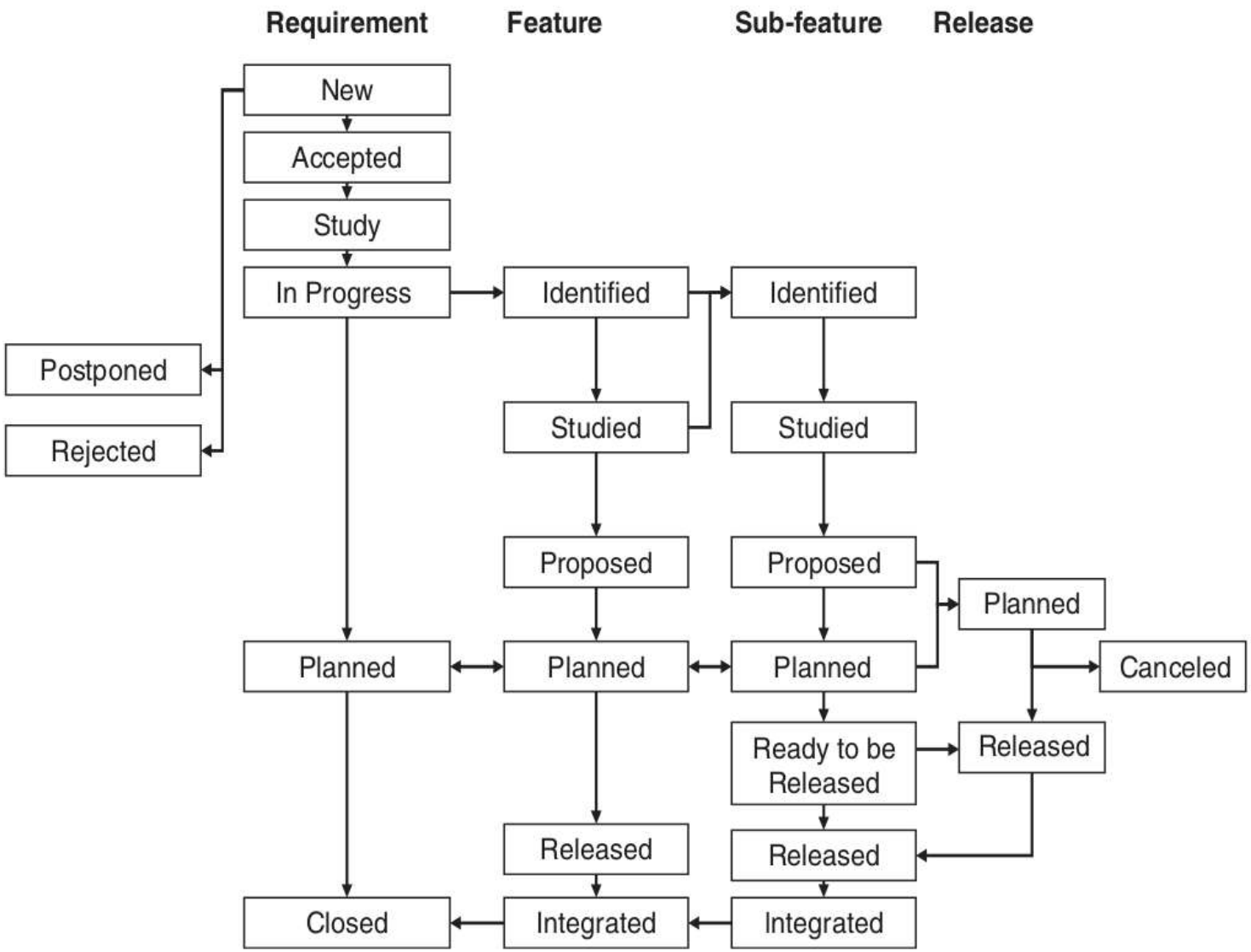
Requirements Management and Paper Specifications IV

- Although some people prefer the expressive richness of natural languages, the ability **to analyze or automate** operations from formal or executable models makes them **the primary choice where possible**. In addition, they are less subject to **unintentional ambiguity and interpretation** than natural language.
- We have also talked about the preference **to describe requirements** rather than executable models because these models **can constrain** the implementation space that can be explored.

Feature Tree



Document Status Hierarchy



ESL Domains

- The design methodologies and languages used for implementation **vary between domains**. Within a subsystem, multiple languages or methodologies may be used.
- ESL Domains are:
  - Dataflow and Control Flow
  - Protocol Stacks
  - Embedded Systems

ESL Domains - Dataflow and Control Flow I

- Within the dataflow domain, we mean such things as modem **receiver and transmitter** data paths, **video processing**, and **signal processing** elements.
- Although dataflow always requires some manner of control flow, early development and analysis of algorithms that define the input-output data transformation usually ignores or **abstracts away the control flow**.
- In early stages of DSP algorithm development, the useful things to specify and model are **static in nature**, such as signal-to-noise ratio in one particular operating mode of an algorithm.

ESL Domains - Dataflow and Control Flow II

- Algorithmic performance modeling is usually done with **sequential programming languages**, such as C/C ++ and MATLAB. System models that connect **algorithm modules** together are also sequential. Because only the flow of data is interesting from one module to another, the connections are usually modeled as **infinitely deep FIFO buffers**.
- **Control flow is about system state and state transformations**. Examples include protocol stacks in wireless communications and user interfaces in all manner of embedded consumer devices, as well as various subsystem control units in automo- tive applications.

ESL Domains - Dataflow and Control Flow III

- Dataflow and control flow together define a system's functionality at a high level. Control flow can be described at **various abstraction levels**, with just the main system states or modes, with and without implementation architecture and partitioning, and with various abstractions of time. For practical purposes, **time and architecture are elemental** for modeling control flow.

ESL Domains - Protocol Stack I

- **Protocol stacks** are subsystems or communication systems that **manage the flow of data on a communications channel** according to the rules of a particular protocol, such as TCP/IP.
- They are called stacks because they are typically designed as **a hierarchy of layers**, each supporting the one above and using the one below.
- The protocol stacks usually identify a **user stratum and a control stratum**. The former deals with data channels carrying payload data and the latter with control messages that are internal to the system.
- The protocol stacks are often layered according to ISO's OSI model [ISO 1994].

ESL Domains - Protocol Stack II

- Protocol stacks are implemented predominantly in **embedded software**, although in some wideband systems like WiMax, **the media access layer** may be implemented at least partially in hardware for performance reasons, and some wireless modems used with personal computers often run the higher protocol layers in the PC's processor.
- **The physical layer** or layer one of a communication system is logically a part of the protocol stack, but when categorizing from an ESL domain perspective, it falls under the **dataflow/control flow domain**.
- The common features of protocol stacks from the ESL modeling point of view are **hard real-time requirements** for processing and communication, **very modular architectures** driven by **standards**, communication that naturally yields to message - passing implementation, and **limited** needs for (instruction level or hardware) **parallelism**.

ESL Domains - Embedded Systems I

- Characteristics:
  - Embedding
  - Real-time properties
  - External environment interaction
  - Nonfunctional constraints
- Embedding - parts or components of other systems
- Real-time properties - Events have a valid lifetime before their effects become invalid
  - Hard real-time constraints
  - Soft real-time constraints
- External environment interaction - a low-level, system-specific set of commands or signals
- It is not based on traditional GUI
- Nonfunctional constraints
  - The physical size

ESL Domains - Embedded Systems II

  - Power consumption
  - Development environments
  - ...
- For “enterprise” systems - those exists but not so relevant
- Much stronger platform modeling is required - especially co-design environments
- Target platforms have to be taken into consideration even during domain modeling

Executable Specifications I

- In an ideal world, a designer of an electronic system would like to have all of the requirements for the system in a form that is **machine-readable** and may be translated into implementation constraints. That is, the requirements would be described in a language that is **formal and thus decipherable by a computer, or executable**, defining input to output transformations of either the system or its environment.
- The executable and formal specifications have three main potential uses:
  1. **Tracing of requirements** through the design process and over the subsystems can be automated.
  2. **Understanding the integration** of heterogeneous systems is improved.
  3. **Implementation and verification** processes can be better **integrated**.

Executable Specifications II

- The executable specification is **a behavioral description of a component or system** object that reflects the particular function and timing of the intended design as seen from the object's interface when executed in a computer simulation.
- The primary purpose of an executable specification is to **verify that the specified behavior of a design entity satisfies the system requirements** when integrated with other components of the system and to verify whether an implementation of the entity is consistent with the specified behavior.
- One caveat on executable specifications should be noted. Because they are executable, **executable specifications are an "implementation"** as well as a manifestation of specification characteristics.

Executable Specifications III

- There are **measurable characteristics that can be derived from the model execution or simulation**; some of these are **relevant** to the system being specified, and some are pure **artifacts** of the executable model. The wise and experienced systems engineer or architect will know how to **separate those characteristics** of an executable specification model that are relevant to the functionality under analysis, from those that are artifacts of the implementation of the model.
- The executable specification in an early design phase is most likely **a simulation model of the system function**. By refining the model and analyzing the different architectural choices, we end up with an executable specification of a real implementation with a good match to the final physical architecture.

Executable Specifications IV

- Using this approach, **integration starts** when building the first **executable specification**. When the final integration phase happens, all functions and interfaces have been verified several times during the design process.

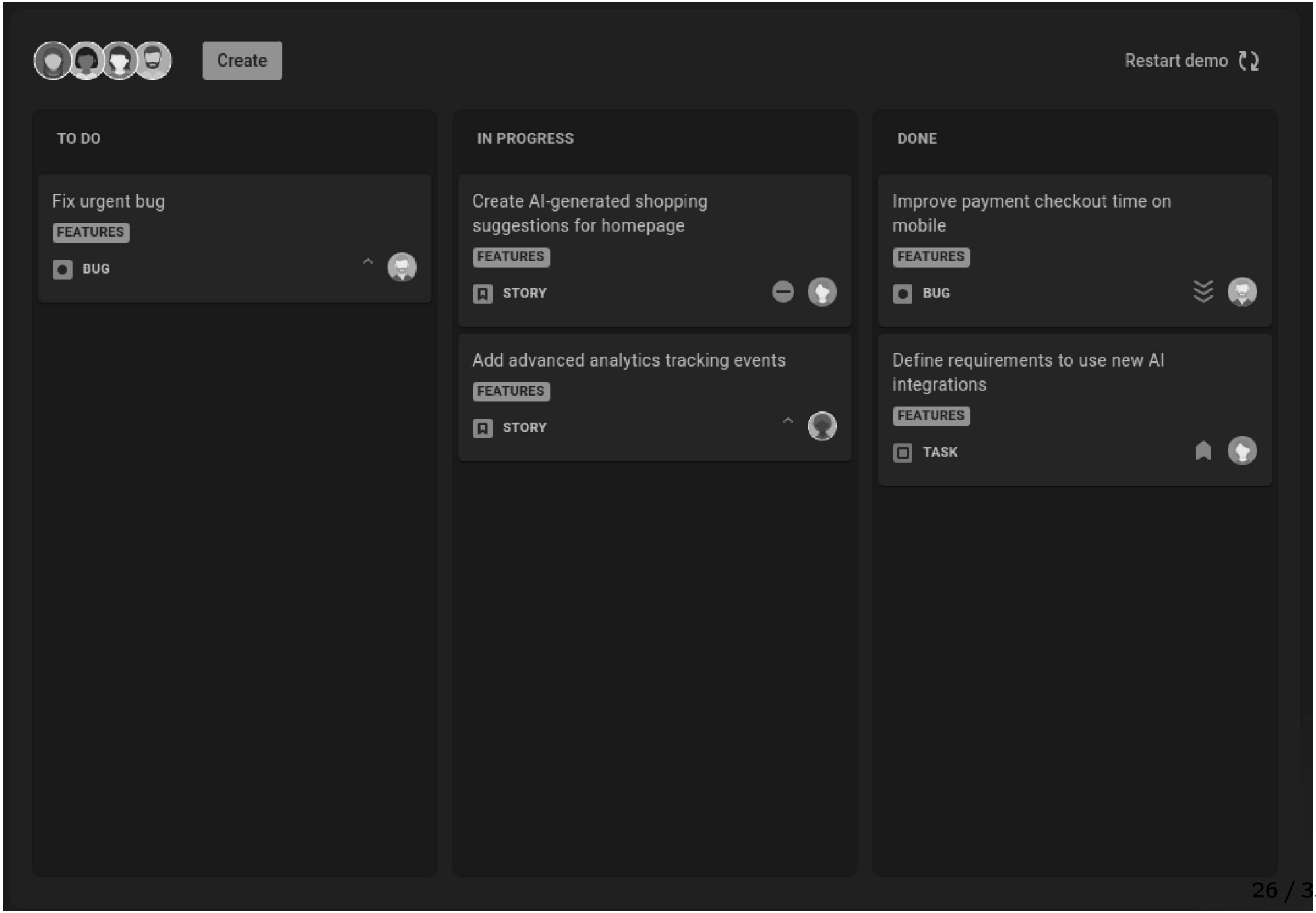
ESL Specification Languages

- There are several ESL specification languages, each with its own advantages and disadvantages.
- Languages used for ESL specification are:
  - python
  - MATLAB
  - SystemC
  - Specification and Description Language (SDL)
  - UML
  - eXtensible Markup Language (XML)

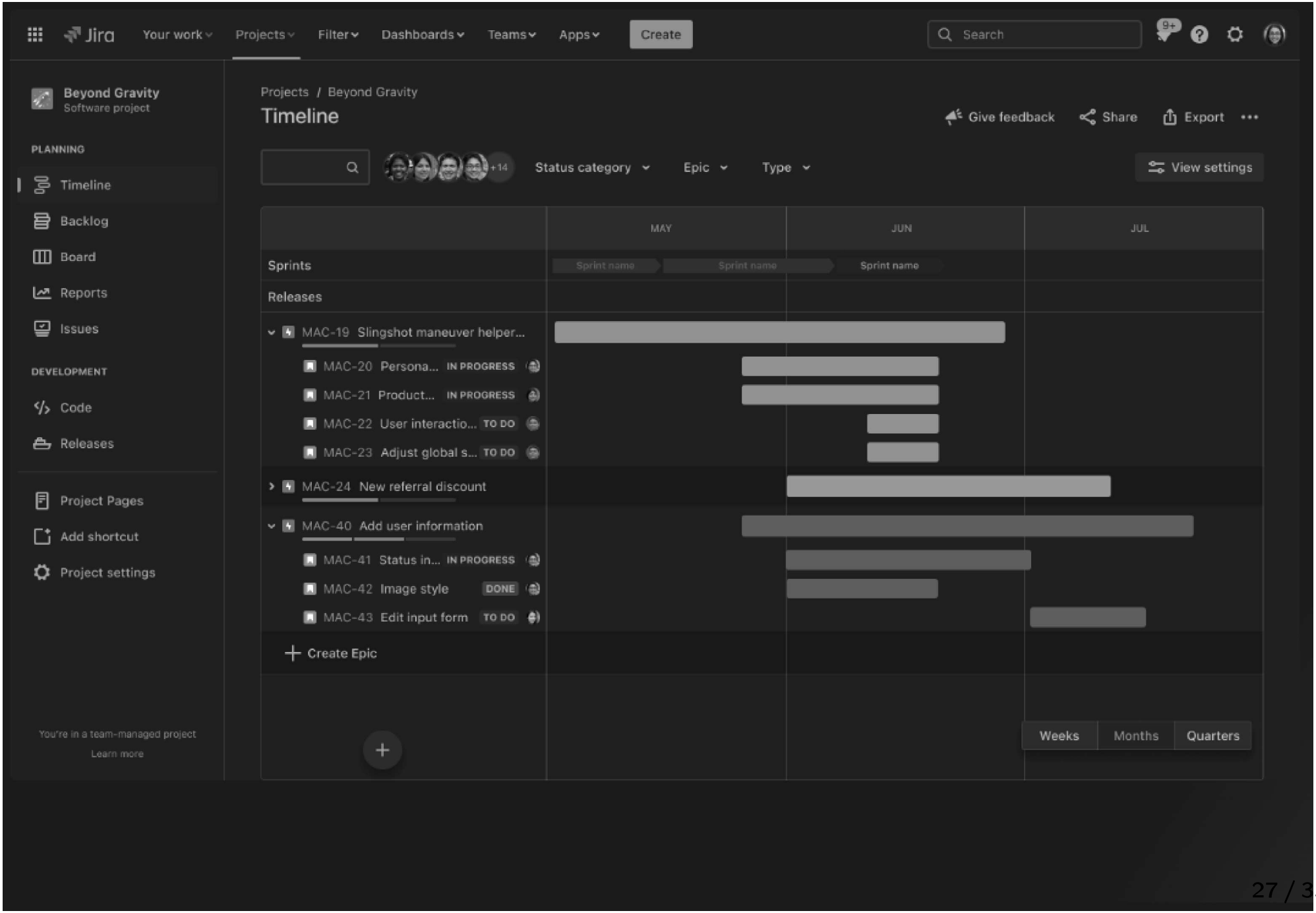
Menagement Software

- IBM Doors
- Atlassian Jira
- Atlassian Confluence

Jira - Plan


Create
Restart demo
TO DO
Fix urgent bug
FEATURES
BUG
IN PROGRESS
Create AI-generated shopping suggestions for homepage
FEATURES
STORY
Add advanced analytics tracking events
FEATURES
STORY
DONE
Improve payment checkout time on mobile
FEATURES
BUG
Define requirements to use new AI integrations
FEATURES
TASK

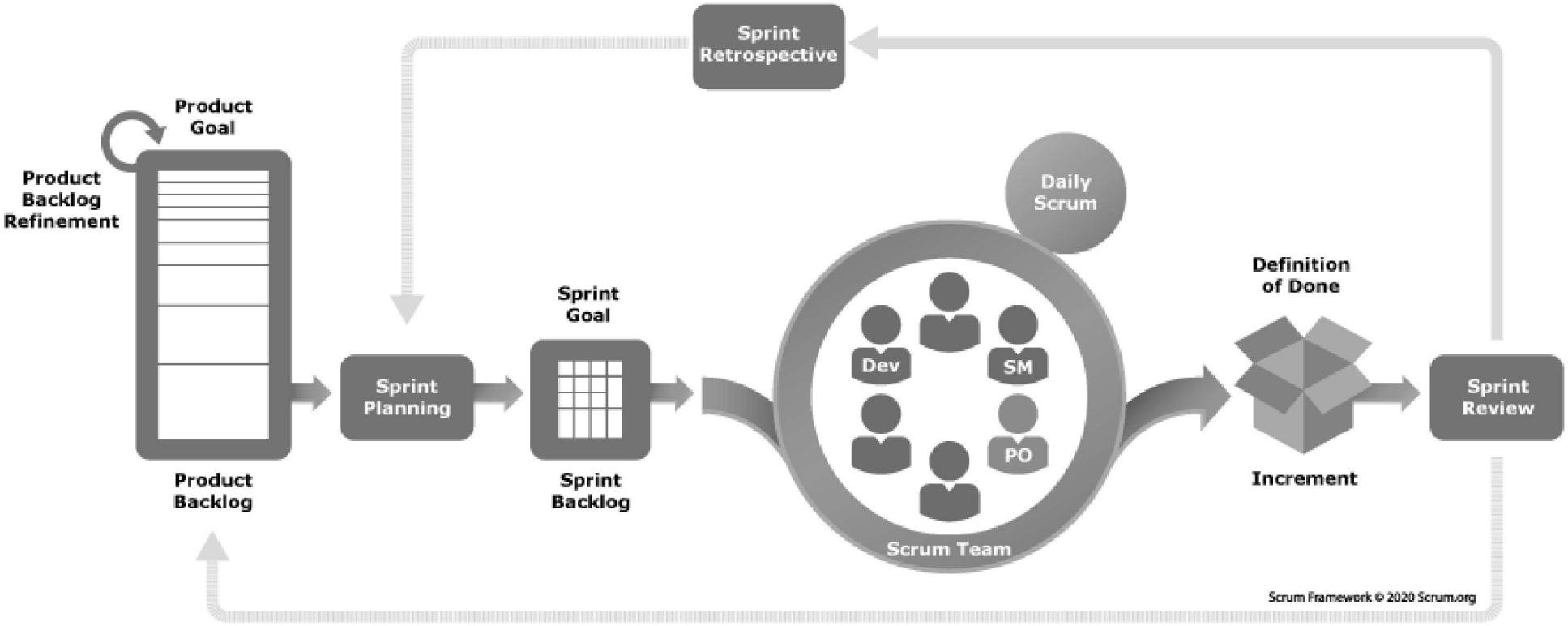
Jira - Track


Jira
Your work
Projects
Filter
Dashboards
Teams
Apps
Create
Search
Beyond Gravity
Software project
PLANNING
Timeline
Backlog
Board
Reports
Issues
DEVELOPMENT
Code
Releases
Project Pages
Add shortcut
Project settings
You're in a team-managed project
Learn more
Projects / Beyond Gravity
Timeline
Give feedback
Share
Export
Status category
Epic
Type
View settings
MAY
JUN
JUL
Sprints
Sprint name
Sprint name
Sprint name
Releases
MAC-19 Slingshot maneuver helper...
MAC-20 Persona... IN PROGRESS
MAC-21 Product... IN PROGRESS
MAC-22 User interactio... TO DO
MAC-23 Adjust global s... TO DO
MAC-24 New referral discount
MAC-40 Add user information
MAC-41 Status in... IN PROGRESS
MAC-42 Image style DONE
MAC-43 Edit input form TO DO
Create Epic
Weeks
Months
Quarters

Team Menagement

- Jira Story Points
- Agile
- Sprint
- Scrum
- Kanban

Scrum


Sprint Retrospective
Product Goal
Product Backlog Refinement
Product Backlog
Sprint Planning
Sprint Goal
Sprint Backlog
Daily Scrum
Dev
SM
PO
Scrum Team
Definition of Done
Increment
Sprint Review
Scrum Framework © 2020 Scrum.org

Example - Code

```
int main (int argc, char **argv)
{
    int samplingStep = 2; // Initial sampling step (default 2)
    int octaves = 4; // Number of analysed octaves (default 4)
    double thres = 4.0; // Blob response treshold
    bool doubleImageSize = false; // Set this flag "true" to double the image
size
    int initLobe = 3; // Initial lobe size, default 3 and 5 (with double image
size)
    int indexSize = 4; // Spatial size of the descriptor window (default 4)
    struct timezone tz; struct timeval tim1, tim2; // Variables for the timing
measure

    // Read the arguments
    ImLoad ImageLoader;
    int arg = 0;
    string fn = "../data/out.surf";
    Image *im=NULL;
    while (++arg < argc) {
        if (! strcmp(argv[arg], "-i"))
            im = ImageLoader.readImage(argv[++arg]);
        if (! strcmp(argv[arg], "-o"))
            fn = argv[++arg];
    }

    // Start measuring the time
    gettimeofday(&tim1, &tz);

    // Create the integral image
    Image iimage(im, doubleImageSize);

    // Start finding the SURF points
    cout << "Finding SURFs...\n";

    // These are the interest points
```

Example - Math

$$\boldsymbol{H}(\boldsymbol{X},\boldsymbol{\sigma}) = \begin{pmatrix} \boldsymbol{L_{xx}}(\boldsymbol{X},\boldsymbol{\sigma}) & \boldsymbol{L_{xy}}(\boldsymbol{X},\boldsymbol{\sigma}) \\ \boldsymbol{L_{xy}}(\boldsymbol{X},\boldsymbol{\sigma}) & \boldsymbol{L_{yy}}(\boldsymbol{X},\boldsymbol{\sigma}) \end{pmatrix}$$

Future - Model Based Development I

- To avoid reimplementing behaviors repeatedly during the modeling process, once behavior at a particular abstraction level is implemented, it may subsequently **be refined but always reused, not reimplemented**. We refer to this as **single sourcing**.
- **Model-based development** replaces the traditional design flow of requirements -> analysis -> design -> implementation -> verify with one of **successive** declarative model **refinement**.
- **Each model is written at a more refined (i.e., less abstract) level than the previous**, and the additional feature level of detail verified. The model is written using a declarative language like UML. This approach **postpones consideration of implementation details** until quite late in the design process, thereby enabling a platform-independent design.

Future - Model Based Development II